Studio Nx

Backend(FastAPI/Python) 23 — GET /api/v1/auth/check-email

2026-10-12

Backend(FastAPI/Python) 23 — 라우터 `GET /api/v1/auth/check-email`

항목	내용
파일	`studionx/backend/app/api/v1/auth.py` (함수 이름 `check_email`), `requirements.txt`
작업	P1-04 회원가입: 이메일 실시간 중복 확인 API (PRD 01 FR-101, FR-108)
작성	Claude (AI), 2026-10-07, 사용자 검토
이전 · 다음	22 `POST /auth/signup` · 24 `GET /auth/check-nickname`

1. 하는 일

주소의 `?email=` 값을 받아 **형식을 검사**하고, 19번 `check_email()`의 결과를 그대로 돌려줍니다.

요청	응답
`GET /api/v1/auth/check-email?email=free@example.com`	200 `{"available": true, "reason": null, "available_at": null}`
`…?email=ROUTE.user@example.COM` (이미 가입됨)	200 `{"available": false, "reason": "TAKEN", …}`
`…?email=gone@example.com` (탈퇴한 지 1일)	200 `{"available": false, "reason": "REJOIN_TOO_SOON", "available_at": "2026-10-09T03:00:00Z"}`
`…?email=abc` (형식이 틀림)	**422** `{"error": {"code": "VALIDATION_ERROR", "details": {"email": "value is not a valid email address: An email address must have an @-sign."}}}`
… (`email`이 없음)	422

2. 언제 쓰나

join.html 이메일 칸 옆의 **Check** 버튼을 누르면 `join.js`(P1-07)가 이 주소로 묻습니다.

응답	이메일 칸 아래 문구 (P1-07)
`available: true`	✓ Available
`TAKEN`	This email is already in use.
`REJOIN_TOO_SOON`	You can sign up again after Oct 9. (`available_at`을 사용자 시간대로 바꿔 보여 줍니다)
422	Enter a valid email address.

형식이 틀리면 422로 응답하는 이유 (2026-10-07 결정): "이미 사용 중"(200, `available: false`)과 "형식이 틀림"(422)을 구분해서 다른 문구를 보여 주기 위해서입니다. 가입(22)도 같은 `EmailStr` 규칙을 쓰므로, 여기서 통과한 이메일은 가입할 때도 형식 때문에 막히지 않습니다.

3. 코드

import (파일 맨 위)

```
from typing import Annotated

from fastapi import APIRouter, Cookie, Query, Response, status
from pydantic import EmailStr

from app.api.deps import Db
from app.schemas.auth import (
    NICKNAME_MAX,
    AvailabilityResponse,
    LoginRequest,
    SignupRequest,
    SignupResponse,
    TokenResponse,
)
from app.schemas.common import ErrorResponse
from app.services import auth_service
```

`auth.py` 맨 위의 import입니다. **`from pydantic import EmailStr`를 이번에 추가했습니다.** 이 함수는 `Annotated`, `Query`, `EmailStr`, `Db`, `AvailabilityResponse`, `auth_service`를 사용합니다.

`requirements.txt`에 한 줄 추가 (새 패키지)

```
email-validator>=2.2       # pydantic EmailStr — 이메일 형식 검사 (P1-04, 2026-10-07)
```

`EmailStr`은 pydantic에 들어 있지만, 실제 검사는 이 패키지가 합니다. 설치된 버전은 2.3.0입니다. 다른 컴퓨터에서는 pull한 뒤 의존성을 다시 설치하면(CLAUDE.md) 자동으로 설치됩니다.

함수

```
@router.get("/check-email", response_model=AvailabilityResponse)
def check_email(db: Db, email: Annotated[EmailStr, Query(max_length=255)]) -> AvailabilityResponse:
    return auth_service.check_email(db, email)
```

템플릿은 `Annotated[str, …]`였는데, `str`을 `EmailStr`로 바꾸고 몸통 한 줄을 채웠습니다.

문법 풀이

import 문법	뜻
`from pydantic import EmailStr`	pydantic의 "이메일 문자열" 타입을 가져옵니다

식	뜻
`@router.get("/check-email", …)`	`GET /api/v1/auth/check-email`에 연결합니다. `GET`은 보기만 하는 요청입니다
`Annotated[EmailStr, Query(max_length=255)]`	이 인자는 **주소의 `?email=`**에서 읽고(`Query`), 255자 이하이며 이메일 형식(`EmailStr`)이어야 합니다. 틀리면 FastAPI가 라우터를 실행하기 전에 422로 응답합니다 (02번 처리기)
`auth_service.check_email(db, email)`	19번을 `release=False`(기본값)로 부릅니다. 그래서 DB를 바꾸지 않습니다
`response_model=AvailabilityResponse`	응답의 모양입니다. `available_at`은 UTC 시각이라서 `…Z`로 나갑니다

`EmailStr`이 받는 것과 막는 것 (직접 확인했습니다)

입력	결과
`kim@naver.com`	통과합니다
`" kim@naver.com "`	통과하고, 앞뒤 공백을 지워 줍니다
`Kim@Naver.COM`	통과하고 `Kim@naver.com`이 됩니다 (도메인만 소문자로 바꾸고, `@` 앞은 17번이 소문자로 바꿉니다)
`abc`, `a b@x.com`, `kim@naver`, `user@localhost`	422
`deleted+85@withdrawn.invalid`	422입니다. `.invalid`는 쓸 수 없는 도메인이므로, **18번에서 비켜 둔 이메일로는 누구도 가입할 수 없습니다**
`한글@naver.com`	통과합니다. 국제 메일 규격(SMTPUTF8)상 올바른 형식이기 때문입니다

4. 흐름

1. FastAPI가 `?email=`을 읽어 길이와 형식을 검사합니다. 틀리면 422로 응답합니다.
2. 19번으로 쓸 수 있는 이메일인지 확인합니다 (DB는 바꾸지 않습니다).
3. 결과를 200으로 돌려줍니다.

5. 왜 이렇게

선택	이유
`GET`으로 확인만 함 (`release=False`)	19번에서 정한 대로입니다. `GET`은 여러 번 불려도 DB가 바뀌면 안 됩니다
"쓸 수 없음"도 200으로 응답	요청 자체는 성공했고, 답이 "아니요"인 것입니다. 오류(4xx)는 요청 자체가 잘못되었을 때(형식이 틀림) 씁니다
형식 검사를 라우터 입구(`EmailStr`)에서 함	틀린 값으로 DB에 묻지 않습니다. 오류 모양도 02번 처리기가 다른 입력 오류와 똑같이 만듭니다
`email-validator` 패키지 추가	정규식으로 흉내 내면 `a..b@x.com`이나 국제 도메인 같은 경우를 놓칩니다. 이메일 규격을 따르는 검사를 씁니다 (2026-10-06 사용자 결정 "EmailStr")

6. 시험 (`tests/test_auth_routes.py`에 10개를 추가해서 모두 16개, 실제 앱과 실제 MySQL로 실행하고 rollback)

시험	확인한 것
없는 이메일	200, `{"available": true, "reason": null, "available_at": null}`
가입된 이메일을 대소문자 섞어서 물음	`TAKEN`
탈퇴한 지 1일	`REJOIN_TOO_SOON`이고, `available_at`이 `Z`(UTC)로 끝납니다
형식이 틀린 5개	`abc`, `a b@x.com`, `kim@naver`, `deleted+85@withdrawn.invalid`, 빈 문자열은 422 `VALIDATION_ERROR`이고, `details`에 `email`이 담깁니다
`email`이 없음	422

시험	확인한 것
탈퇴한 지 5일 된 행이 있을 때	`available: true`이지만, 그 행의 이메일은 **그대로**입니다 (GET은 확인만 합니다)

전체 시험 159개가 통과했고, `ruff` 검사도 통과했습니다.